C Program Control

4

Objectives

In this chapter, you'll learn:

- The essentials of counter-controlled iteration.
- To use the `for` and `do...while` iteration statements to execute statements repeatedly.
- To understand multiple selection using the `switch` selection statement.
- To use the `break` and `continue` statements to alter the flow of control.
- To use the logical operators to form complex conditional expressions in control statements.
- To avoid the consequences of confusing the equality and assignment operators.

turns `0` indicating that it was unable to read the input as an integer. In this case, the variable `grade` does *not* receive a value.

Function `scanf` can read multiple inputs, as in

```
scanf("%d%d", &number1, &number2); // read two integers
```

If the input is successful, `scanf` will return `2`, indicating that two values were read. If the user enters a string for the first value, `scanf` will return `0` and neither `number1` nor `number2` will receive a value. If the user enters an integer followed by a string, `scanf` will return `1` and only `number1` will receive a value.



Error-Prevention Tip 4.10

To make your input processing more robust, check `scanf`'s return value to ensure that the number of inputs read matches the number of inputs expected. Otherwise, your program will use the values of the variables as if `scanf` completed successfully. This could lead to logic errors, program crashes or even attacks.

Range Checking

Even if a `scanf` operates successfully, the values read might still be *invalid*. For example, grades are typically integers in the range 0–100. In a program that inputs such grades, you should **validate** the grades by using **range checking** to ensure that they are values from 0 to 100. You can then ask the user to reenter any value that's out of range. If a program requires inputs from a specific set of values (e.g., nonsequential product codes), you can ensure that each input matches a value in the set. For more information, see Chapter 5, "Integer Security," of Robert Seacord's book *Secure Coding in C and C++, 2/e*.

Summary

Section 4.2 Iteration Essentials

- Most programs involve iteration, or looping. A loop is a group of instructions the computer executes repeatedly while some **loop-continuation condition** (p. 146) remains true.
- **Counter-controlled iteration** is sometimes called **definite iteration** (p. 146) because we know in advance exactly how many times the loop will execute.
- **Sentinel-controlled iteration** is sometimes called **indefinite iteration** (p. 146) because it's not known in advance how many times the loop will execute; the loop includes statements that obtain data each time the loop is performed.
- In counter-controlled iteration, a **control variable** (p. 146) is used to count the number of iterations. The control variable is **incremented** (or **decremented**) each time the group of instructions is performed (p. 147). When the correct number of iterations has been performed, the loop terminates, and the program resumes execution with the statement after the iteration statement.
- The **sentinel value** indicates "end of data." The sentinel is entered after all regular data items have been supplied to the program. Sentinels must be distinct from regular data items.

Section 4.3 Counter-Controlled Iteration

- Counter-controlled iteration requires the **name** (p. 147) of a control variable (or loop counter), the **initial value** (p. 147) of the control variable, the **increment** (or **decrement**) by which the control variable is modified each time through the loop, and the condition that tests for the **final value** (p. 147) of the control variable (i.e., whether looping should continue).

Section 4.4 `for` *Iteration Statement*

- The **`for` iteration statement** handles all the details of counter-controlled iteration.
- When the `for` statement begins executing, its control variable is initialized. Then, the loop-continuation condition is checked. If the condition is true, the loop's body executes. The control variable is then incremented, and the loop begins again with the loop-continuation condition. This process continues until the loop-continuation condition fails.
- The general format of the `for` statement is

```
for (initialization; condition; increment) {
   statements
}
```

  where the *initialization* expression initializes (and possibly defines) the control variable, the *condition* expression is the loop-continuation condition, and the *increment* expression increments the control variable.
- The **comma operator** (p. 150) guarantees that lists of expressions evaluate from left to right. The value of the entire expression is that of the rightmost expression.
- The three expressions in the `for` statement are optional. If the *condition* expression is omitted, C assumes that the condition is true, thus creating an infinite loop. One might omit the *initialization* expression if the control variable is initialized before the loop. The *increment* expression might be omitted if the increment is calculated by statements in the `for` statement's body or if no increment is needed.
- The *increment* expression in the `for` statement acts like a standalone C statement at the end of the body of the `for`.
- The two semicolons in the `for` statement are required.

Section 4.5 `for` *Statement: Notes and Observations*

- The initialization, loop-continuation condition and increment can contain arithmetic expressions.
- The "increment" may be negative (in which case it's really a decrement and the loop actually counts downward).
- If the loop-continuation condition is initially false, the body portion of the loop isn't performed. Instead, execution proceeds with the statement following the `for` statement.

Section 4.6 Examples Using the `for` *Statement*

- **Function `pow`** (p. 154) performs exponentiation. The function `pow(x, y)` calculates the value of `x` raised to the `y`th power. It takes two arguments of type `double` and returns a `double` value.
- Type **`double`** is a floating-point type much like `float`, but typically a variable of type `double` can store a value of much **greater magnitude** with **greater precision** than `float`.
- The **header `<math.h>`** (p. 154) should be included whenever a math function such as `pow` is used.
- The conversion specifier `%21.2f` denotes that a floating-point value will be displayed **right justified** in a field of 21 characters with two digits to the right of the decimal point.
- To **left justify** a value in a field, place a - (minus sign) between the `%` and the field width.

Section 4.7 `switch` *Multiple-Selection Statement*

- Occasionally, an algorithm will contain a series of decisions in which a variable or expression is tested separately for each of the constant integral values it may assume, and different actions are taken. This is called **multiple selection**. C provides the **`switch` statement** to handle this.
- The `switch` statement consists of a series of **`case` labels** (p. 159), an optional **`default` case** and statements to execute for each case.

- The **getchar function** (from the standard input/output library) reads and returns as an `int` one character from the keyboard.
- Characters are normally stored in variables of type **char** (p. 157). Characters can be stored in any integer data type because they're usually represented as one-byte integers in the computer. Thus, we can treat a character as either an integer or a character, depending on its use.
- Many computers today use the **ASCII** (**American Standard Code for Information Interchange**; p. 158) character set in which 97 represents the lowercase letter `'a'`.
- Characters can be read with `scanf` by using the conversion specifier **%c**.
- **Assignment expressions** as a whole actually **have a value**. This value is assigned to the variable on the left side of the `=`.
- The fact that assignment statements have values can be useful for setting several variables to the same value, as in `a = b = c = 0;`.
- **EOF** is often used as a sentinel value. `EOF` is a symbolic integer constant defined in `<stdio.h>`.
- On Linux/UNIX systems and many others, the `EOF` indicator is entered by typing *<Ctrl> d*. On other systems, such as Microsoft Windows, the `EOF` indicator can be entered by typing *<Ctrl> z*.
- Keyword `switch` is followed by the **controlling expression** (p. 159) in parentheses. The value of this expression is compared with each of the `case` labels. If a match occurs, the statements for that `case` execute. If no match occurs, the `default` case executes.
- The **break statement** causes program control to continue with the statement after the `switch`. The `break` statement prevents the `case`s in a `switch` statement from running together.
- Each `case` can have one or more actions. The `switch` statement is different from all other control statements in that braces are not required around multiple actions in a `case` of a `switch`.
- **Listing several case labels together** simply means that the same set of actions is to occur for any of these `case`s.
- Remember that the `switch` statement can be used only for testing a **constant integral expression** (p. 161)—i.e., any combination of character constants and integer constants that evaluates to a constant integer value. A character constant can be represented as the specific character in single quotes, such as `'A'`. Characters must be enclosed within single quotes to be recognized as character constants. Integer constants are simply integer values.
- In addition to integer types `int` and `char`, C provides types **short int** (which can be abbreviated as **short**) and **long int** (which can be abbreviated as **long**), as well as **unsigned** versions of all the integral types. The C standard specifies the minimum value range for each type, but the actual range may be greater, depending on the implementation. For `short` `int`s the minimum range is –32767 to +32767. The minimum range of values for `long` `int`s is –2147483647 to +2147483647. The range of values for an `int` is greater than or equal to that of a `short` `int` and less than or equal to that of a `long` `int`. On many of today's platforms, `int`s and `long` `int`s represent the same range of values. The data type `signed` `char` can be used to represent integers in the range –127 to +127 or any of the characters in the computer's character set. See Section 5.2.4.2 of the C standard document for the complete list of `signed` and `unsigned` integer-type ranges.

*Section 4.8 **do...while** Iteration Statement*

- The **do...while statement** tests the loop-continuation condition *after* the loop body is performed. Therefore, the loop body executes at least once. When a `do...while` terminates, execution continues with the statement after the `while` clause.

*Section 4.9 **break** and **continue** Statements*

- The **break statement**, when executed in a `while`, `for`, `do...while` or `switch` statement, causes immediate exit from that statement. Program execution continues with the next statement.

- The **`continue` statement**, when executed in a `while`, `for` or `do...while` statement, skips the remaining statements in the body and performs the next loop iteration. In `while` and `do...while`, the loop-continuation test is evaluated immediately after the `continue` statement is executed. In a `for`, the increment expression is executed, then the loop-continuation test is evaluated.

Section 4.10 Logical Operators

- **Logical operators** may be used to form complex conditions by combining simple conditions. The logical operators are `&&` (logical AND), `||` (logical OR) and `!` (logical NOT, or logical negation).
- A condition containing the **`&&`** (**logical AND**; p. 164) operator is true if and only if both of the simple conditions are true.
- C evaluates all expressions that include relational operators, equality operators, and/or logical operators to 0 or 1. Although C sets a **true value** to **1**, it accepts ***any* nonzero** value as **true**.
- A condition containing the **`||`** (**logical OR**; p. 164) operator is true if either or both of the simple conditions are true.
- The `&&` operator has a higher precedence than `||`. Both operators associate from left to right.
- An expression containing `&&` or `||` operators is evaluated only until truth or falsehood is known.
- C provides the **`!`** (**logical negation**; p. 164) operator to enable you to "reverse" the meaning of a condition. Unlike the binary operators `&&` and `||`, which combine two conditions, the unary logical negation operator has only a single condition as an operand.
- The logical negation operator is placed before a condition when we're interested in choosing a path of execution if the original condition (without the logical negation operator) is false.
- In most cases, you can avoid using logical negation by expressing the condition differently with an appropriate relational operator.

Section 4.11 Confusing Equality (==) and Assignment (=) Operators

- Programmers often accidentally swap the operators `==` (equality) and `=` (assignment). What makes these swaps so damaging is that they do not ordinarily cause syntax errors. Rather, statements with these errors ordinarily compile correctly, allowing programs to run to completion while likely generating incorrect results through runtime logic errors.
- You may be inclined to write conditions such as `x == 7` with the variable name on the left and the constant on the right. By reversing these terms so that the constant is on the left and the variable name is on the right, as in `7 == x`, then if you accidentally replace the `==` operator with `=`, you'll be protected by the compiler. The compiler will treat this as a syntax error, because only a variable name can be placed on the left-hand side of an assignment statement.
- Variable names are said to be ***lvalues*** (for "**left values**"; p. 168) because they *can* be used on the left side of an assignment operator.
- Constants are said to be ***rvalues*** (for "**right values**"; p. 168) because they can be used *only* on the right side of an assignment operator. *lvalues* can also be used as *rvalues*, but not vice versa.

Self-Review Exercises

4.1 Fill in the blanks in each of the following statements.

a) Counter-controlled iteration is also known as _____ iteration because it's known in advance how many times the loop will be executed.

b) Sentinel-controlled iteration is also known as _____ iteration because it's not known in advance how many times the loop will be executed.

c) In counter-controlled iteration, a(n) _____ is used to count the number of times a group of instructions should be repeated.

d) The _____ statement, when executed in an iteration statement, causes the next iteration of the loop to be performed immediately.
e) The _____ statement, when executed in an iteration statement or a `switch`, causes an immediate exit from the statement.
f) The _____ is used to test a particular variable or expression for each of the constant integral values it may assume.

4.2 State whether the following are *true* or *false*. If the answer is *false*, explain why.
a) The `default` case is required in the `switch` selection statement.
b) The `break` statement is required in the `default` case of a `switch` selection statement.
c) The expression `(x > y && a < b)` is true if either `x > y` is true or `a < b` is true.
d) An expression containing the `||` operator is true if either or both of its operands is true.

4.3 Write a statement or a set of statements to accomplish each of the following tasks:
a) Sum the odd integers between 1 and 99 using a `for` statement. Use the unsigned integer variables `sum` and `count`.
b) Print the value `333.546372` in a field width of `15` characters with precisions of `1`, `2`, `3`, `4` and `5`. Left justify the output. What are the five values that print?
c) Calculate the value of `2.5` raised to the power of `3` using the `pow` function. Print the result with a precision of `2` in a field width of `10` positions. What is the value that prints?
d) Print the integers from 1 to 20 using a `while` loop and the counter variable `x`. Print only five integers per line. [*Hint:* Use the calculation `x % 5`. When the value of this is 0, print a newline character, otherwise print a tab character.]
e) Repeat Exercise 4.3(d) using a `for` statement.

4.4 Find the error in each of the following code segments and explain how to correct it.
a)
```
x = 1;
while (x <= 10);
   ++x;
}
```
b)
```
for (double y = .1; y != 1.0; y += .1) {
   printf("%f\n", y);
}
```
c)
```
switch (n) {
   case 1:
      puts("The number is 1");
   case 2:
      puts("The number is 2");
      break;
   default:
      puts("The number is not 1 or 2");
      break;
   }
```
d) The following code should print the values 1 to 10.
```
n = 1;

while (n < 10) {
  printf("%d ", n++);
}
```

Answers to Self-Review Exercises

4.1 a) definite. b) indefinite. c) control variable or counter. d) `continue`. e) `break`. f) `switch` selection statement.

4.2 a) False. The `default` case is optional. If no default action is needed, then there's no need for a `default` case.

b) False. The `break` statement is used to exit the `switch` statement. The `break` statement is not required in *any* `case`.

c) False. Both of the relational expressions must be true in order for the entire expression to be true when using the `&&` operator.

d) True.

4.3 a)
```
unsigned int sum = 0;
for (unsigned int count = 1; count <= 99; count += 2) {
   sum += count;
}
```

b)
```
printf("%-15.1f\n", 333.546372); // prints 333.5
printf("%-15.2f\n", 333.546372); // prints 333.55
printf("%-15.3f\n", 333.546372); // prints 333.546
printf("%-15.4f\n", 333.546372); // prints 333.5464
printf("%-15.5f\n", 333.546372); // prints 333.54637
```

c)
```
printf("%10.2f\n", pow(2.5, 3)); // prints 15.63
```

d)
```
unsigned int x = 1;
while (x <= 20) {
   printf("%d", x);
   if (x % 5 == 0) {
      puts("");
   }
   else {
      printf("%s", "\t");
   }
   ++x;
}
```

or

```
unsigned int x = 1;
while (x <= 20) {
   if (x % 5 == 0) {
      printf("%u\n", x++);
   }
   else {
      printf("%u\t", x++);
   }
}
```

or

```
unsigned int x = 0;
while (++x <= 20) {
   if (x % 5 == 0) {
      printf("%u\n", x);
   }
   else {
      printf("%u\t", x);
   }
}
```

e)
```
for (unsigned int x = 1; x <= 20; ++x) {
   printf("%u", x);
   if (x % 5 == 0) {
      puts("");
   }
   else {
      printf("%s", "\t");
   }
}
```

or

```
for (unsigned int x = 1; x <= 20; ++x) {
   if (x % 5 == 0) {
      printf("%u\n", x);
   }
   else {
      printf("%u\t", x);
   }
}
```

4.4 a) Error: The semicolon after the `while` header causes an infinite loop.
Correction: Replace the semicolon with a `{` or remove both the `;` and the `}`.

b) Error: Using a floating-point number to control a `for` iteration statement.
Correction: Use an integer, and perform the proper calculation to get the values you desire.

```
for (int y = 1; y != 10; ++y) {
   printf("%f\n", (float) y / 10);
}
```

c) Error: Missing `break` statement in the statements for the first `case`.
Correction: Add a `break` statement at the end of the statements for the first `case`. This is not necessarily an error if you want the statement of `case 2:` to execute every time the `case 1:` statement executes.

d) Error: Improper relational operator used in the `while` iteration-continuation condition.
Correction: Use `<=` rather than `<`.

Exercises

4.5 Find the error in each of the following. (*Note:* There may be more than one error.)

a)
```
for (a = 25, a <= 1, a--); {
   printf("%d\n", a);
}
```

b) The following code should print whether a given integer is odd or even:

```
switch (value) {
   case (value % 2 == 0):
       puts("Even integer");
   case (value % 2 != 0):
       puts("Odd integer");
}
```

c) The following code should calculate incremented salary after 10 years:

```
for (int year = 1; year <= 10; ++year) {
    double salary += salary * 0.05;
}

printf("%4u%21.2f\n", year, salary);
```

d)
```
for (double y = 7.11; y != 7.20; y += .01)
 printf("%7.2f\n", y);
```

e) The following code should output all multiples of 3 from 1 to 100:

```
for (int x = 3; x <= 100; x%3 == 0; x++ ) {
  printf("%d\n", x);
}
```

f)
```
x = 1;
while ( x <= 10 ) {

     printf("%d\n", x);

}
```

g) The following code should sum the squares of all numbers from 1 to 50 (assume `sum` is initialized to 0):

```
for (x = 1; x == 50; ++x) {
  sum =+ x * x;
}
```

4.6 State which values of the control variable `x` are printed by each of the following `for` statements:

a)
```
for (x = 20; x >= 3; x -= 3) {
  printf("%u\n", x);
}
```

b)
```
for (x = 7; x <= 27; x += 5) {
  printf("%u\n", x);
}
```

c)
```
for (x = 2; x <= 20; x += 4) {
  printf("%u\n", x);
}
```

d)
```
for (x = 30; x >= 15; x -= 6) {
  printf("%u\n", x);
}
```

e)
```
for (x = 22; x >= 2; x -= 5) {
  printf("%d\n", x);
}
```

4.7 Write `for` statements that print the following sequences of values:

a) 1, 3, 5, 7, 9, 11, 13
b) 2, 5, 8, 11, 14, 17
c) 30, 20, 10, 0, –10, –20, –30
d) 15, 23, 31, 39, 47, 55

4.8 What does the following program do?

```
#include <stdio.h>
int main(void)
{
   int x, i, j;
   // prompt user for input
   printf("%s", "Enter an integer in the range 1-20:");
   scanf("%d", &x); // read values for x
   for (i = 1; i <= x; i++) { // count from 1 to x
      for (j = 1; j <= x; j++) { // count from 1 to x
         if (j==i)
            printf("%c", '@'); // output @
         else
            printf(" ");
      } // end inner for
      printf("\n");
   } // end outer for
} // end of main
```

4.9 ***(Sum and Average of Integers)*** Write a program to sum a sequence of integers and calculate their average. Assume that the first integer read with `scanf` specifies the number of values to be entered. Your program should read only one value each time `scanf` is executed. A typical input sequence might be

```
7 678 234 315 489 536 456 367
```

where the `7` indicates that the subsequent `7` values are to be summed.

4.10 ***(Conversion Celsius to Fahrenheit)*** Write a program that converts temperatures from 30°C to 50°C to the Fahrenheit scale. The program should print a table displaying temperatures in the two scales side by side. [*Hint*: $°F = \frac{9}{5}C + 32$]

4.11 ***(Calculating the Sum of Multiples)*** Write a program to calculate and print the sum of all multiples of 7 from 1 to 100.

4.12 ***(Prime Numbers)*** Write a program to calculate and print a list of all prime numbers from 1 to 100.

4.13 ***(Natural Numbers Arithmetic)*** Write a program that prints the sum, the sum of the squares, and the sum of the cubes of all natural numbers from 1 till any number entered by the user.

4.14 ***(Factorials)*** The *factorial* function is used frequently in probability problems. The factorial of a positive integer n (written $n!$ and pronounced "n factorial") is equal to the product of the positive integers from 1 to n. Write a program that evaluates the factorials of the integers from 1 to 5. Print the results in tabular format. What difficulty might prevent you from calculating the factorial of 20?

4.15 ***(Modified Compound-Interest Program)*** Modify the compound-interest program of Section 4.6 to repeat its steps for interest rates of 5%, 6%, 7%, 8%, 9%, and 10%. Use a `for` loop to vary the interest rate.

4.16 ***(Triangle-Printing Program)*** Write a program that prints the following patterns separately, one below the other. Use `for` loops to generate the patterns. All asterisks (`*`) should be printed by a

single `printf` statement of the form `printf("%s", "*");` (this causes the asterisks to print side by side). [*Hint:* The last two patterns require that each line begin with an appropriate number of blanks.]

```
(A)          (B)          (C)          (D)
*            **********   **********          *
**           *********    *********          **
***          ********     ********          ***
****         *******      *******          ****
*****        ******       ******          *****
******       *****        *****          ******
*******      ****         ****          *******
********     ***          ***          ********
*********    **           **          *********
**********   *            *          **********
```

4.17 ***(Calculating Credit Limits)*** Collecting money becomes increasingly difficult during periods of recession, so companies may tighten their credit limits to prevent their accounts receivable (money owed to them) from becoming too large. In response to a prolonged recession, one company has cut its customers' credit limits in half. Thus, if a particular customer had a credit limit of $2000, it's now $1000. If a customer had a credit limit of $5000, it's now $2500. Write a program that analyzes the credit status of three customers of this company. For each customer you're given:

a) The customer's account number.
b) The customer's credit limit before the recession.
c) The customer's current balance (i.e., the amount the customer owes the company).

Your program should calculate and print the new credit limit for each customer and should determine (and print) which customers have current balances that exceed their new credit limits.

4.18 ***(Bar-Chart Printing Program)*** One interesting application of computers is drawing graphs and bar charts. Write a program that reads five numbers (each between 1 and 30). For each number read, your program should print a line containing that number of adjacent asterisks. For example, if your program reads the number seven, it should print *******.

4.19 ***(Calculating Sales)*** An online retailer sells five different products whose retail prices are shown in the following table:

Product number	Retail price
1	$ 2.98
2	$ 4.50
3	$ 9.98
4	$ 4.49
5	$ 6.87

Write a program that reads a series of pairs of numbers as follows:

a) Product number
b) Quantity sold for one day

Your program should use a `switch` statement to help determine the retail price for each product. Your program should calculate and display the total retail value of all products sold last week.

4.20 *(Truth Tables)* Complete the following truth tables by filling in each blank with 0 or 1.

Condition1	Condition2	Condition1 && Condition2
0	0	0
0	nonzero	0
nonzero	0	_____
nonzero	nonzero	_____

Condition1	Condition2	Condition1 || Condition2
0	0	0
0	nonzero	1
nonzero	0	_____
nonzero	nonzero	_____

Condition1	!Condition1
0	1
nonzero	_____

4.21 *(ASCII Values)* Write a program to convert and print the characters for the ASCII values 0 to 127. The program should print 10 characters per line.

4.22 *(Average Grade)* Modify the program of Fig. 4.7 so that it calculates the average grade for the class.

4.23 *(Calculating the Compound Interest with Integers)* Modify the program of Fig. 4.6 so that it uses only integers to calculate the compound interest. [*Hint:* Treat all monetary amounts as integral numbers of pennies. Then "break" the result into its dollar portion and cents portion by using the division and remainder operations, respectively. Insert a period.]

4.24 Assume `i = 5`, `j = 7`, `k = 4` and `m = -2`. What does each of the following statements print?

a) `printf("%d", i == 5);`
b) `printf("%d", j!= 3);`
c) `printf("%d", i >= 5 && j < 4);`
d) `printf("%d", !m && k > m);`
e) `printf("%d", !k || m);`
f) `printf("%d", k - m < j || 5 - j >= k);`
g) `printf("%d", j + m <= i && !0);`
h) `printf("%d", !(j - m));`
i) `printf("%d", !(k > m));`
j) `printf("%d", !(j > k));`

4.25 *(Table of Decimal, Binary, Octal and Hexadecimal Equivalents)* Write a program that prints a table of the binary, octal and hexadecimal equivalents of the decimal numbers in the range

1 through 256. If you're not familiar with these number systems, read Appendix C before you attempt this exercise. [*Note:* You can display an integer as an octal or hexadecimal value with the conversion specifiers `%o` and `%X`, respectively.]

4.26 ***(Calculating the Value of π)*** Calculate the value of π from the infinite series

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \cdots$$

Print a table that shows the value of π approximated by one term of this series, by two terms, by three terms, and so on. How many terms of this series do you have to use before you first get 3.14? 3.141? 3.1415? 3.14159?

4.27 ***(Pythagorean Triples)*** A right triangle can have sides that are all integers. The set of three integer values for the sides of a right triangle is called a Pythagorean triple. These three sides must satisfy the relationship that the sum of the squares of two of the sides is equal to the square of the hypotenuse. Find all Pythagorean triples for side1, side2, and the hypotenuse all no larger than 500. Use a triple-nested `for` loop that simply tries all possibilities. This is an example of "brute-force" computing. It's not aesthetically pleasing to many people. But there are many reasons why these techniques are important. First, with computing power increasing at such a phenomenal pace, solutions that would have taken years or even centuries of computer time to produce with the technology of just a few years ago can now be produced in hours, minutes or even seconds. Recent microprocessor chips can process a billion instructions per second! Second, as you'll learn in more advanced computer science courses, there are large numbers of interesting problems for which there's no known algorithmic approach other than sheer brute force. We investigate many kinds of problem-solving methodologies in this book. We'll consider many brute-force approaches to various interesting problems.

4.28 ***(Calculating Weekly Pay)*** A company pays its employees as managers (who receive a fixed weekly salary), hourly workers (who receive a fixed hourly wage for up to the first 40 hours they work and "time-and-a-half"—i.e., 1.5 times their hourly wage—for overtime hours worked), commission workers (who receive $250 plus 5.7% of their gross weekly sales), or pieceworkers (who receive a fixed amount of money for each of the items they produce—each pieceworker in this company works on only one type of item). Write a program to compute the weekly pay for each employee. You do not know the number of employees in advance. Each type of employee has its own pay code: Managers have paycode 1, hourly workers have code 2, commission workers have code 3 and pieceworkers have code 4. Use a `switch` to compute each employee's pay based on that employee's paycode. Within the `switch`, prompt the user (i.e., the payroll clerk) to enter the appropriate facts your program needs to calculate each employee's pay based on that employee's paycode. [*Note:* You can input values of type `double` using the conversion specifier `%lf` with `scanf`.]

4.29 ***(De Morgan's Laws)*** In this chapter, we discussed the logical operators `&&`, `||`, and `!`. De Morgan's Laws can sometimes make it more convenient for us to express a logical expression. These laws state that the expression `!(`*condition1* `&&` *condition2*`)` is logically equivalent to the expression `(!`*condition1* `||` `!`*condition2*`)`. Also, the expression `!(`*condition1* `||` *condition2*`)` is logically equivalent to the expression `(!`*condition1* `&&` `!`*condition2*`)`. Use De Morgan's Laws to write equivalent expressions for each of the following, and then write a program to show that both the original expression and the new expression in each case are equivalent.

a) `!(x < 5) && !(y >= 7)`
b) `!(a == b) || !(g != 5)`
c) `!((x <= 8) && (y > 4))`
d) `!((i > 4) || (j <= 6))`

4.30 ***(Replacing `switch` with `if...else`)*** Rewrite the program of Fig. 4.7 by replacing the `switch` statement with a nested `if...else` statement; be careful to deal with the `default` case properly. Then

rewrite this new version by replacing the nested `if...else` statement with a series of `if` statements; here, too, be careful to deal with the `default` case properly (this is more difficult than in the nested `if...else` version). This exercise demonstrates that `switch` is a convenience and that any `switch` statement can be written with only single-selection statements.

4.31 ***(Diamond-Printing Program)*** Write a program that prints the following diamond shape. You may use `printf` statements that print either a single asterisk (`*`) or a single blank. Maximize your use of iteration (with nested `for` statements) and minimize the number of `printf` statements.

```
   *
  ***
 *****
*******
*********
*******
 *****
  ***
   *
```

4.32 ***(Modified Diamond-Printing Program)*** Modify the program you wrote in Exercise 4.31 to read an odd number in the range 1 to 19 to specify the number of rows in the diamond. Your program should then display a diamond of the appropriate size.

4.33 ***(Roman-Numeral Equivalent of Decimal Values)*** Write a program that prints a table of all the Roman-numeral equivalents of the decimal numbers in the range 1 to 100.

4.34 Describe the process you would use to replace a `do...while` loop with an equivalent `while` loop. What problem occurs when you try to replace a `while` loop with an equivalent `do...while` loop? Suppose you've been told that you must remove a `while` loop and replace it with a `do...while`. What additional control statement would you need to use and how would you use it to ensure that the resulting program behaves exactly as the original?

4.35 A criticism of the `break` statement and the `continue` statement is that each is unstructured. Actually, `break` statements and `continue` statements can always be replaced by structured statements, although doing so can be awkward. Describe in general how you would remove any `break` statement from a loop in a program and replace that statement with some structured equivalent. [*Hint:* The `break` statement leaves a loop from within the body of the loop. The other way to leave is by failing the loop-continuation test. Consider using in the loop-continuation test a second test that indicates "early exit because of a 'break' condition."] Use the technique you developed here to remove the `break` statement from the program of Fig. 4.11.

4.36 What does the following program segment do?

```
1  int n = 4, a = 1;
2     int i, c;
3        for (i = 1; i <= n; i++) {
4
5           for (c = 1; c <= i; c++) {
6
7              printf("%d", a);
8              a++;
9           }
10    printf("\n");
11 }
```

4.37 Describe in general how you would remove any `continue` statement from a loop in a program and replace that statement with some structured equivalent. Use the technique you developed here to remove the `continue` statement from the program of Fig. 4.12.

4.38 ***("The Twelve Days of Christmas" Song)*** Write a program that uses iteration and `switch` statements to print the song "The Twelve Days of Christmas." One `switch` statement should be used to print the day (i.e., "first," "second," etc.). A separate `switch` statement should be used to print the remainder of each verse.

4.39 ***(Limitations of Floating-Point Numbers for Monetary Amounts)*** Section 4.6 cautioned about using floating-point values for monetary calculations. Try this experiment: Create a `float` variable with the value `1000000.00`. Next add to that variable the literal `float` value `0.12f`. Display the result using `printf` and the conversion specifier `"%.2f"`. What do you get?

Making a Difference

4.40 ***(World Population Growth)*** World population has grown considerably over the centuries. Continued growth could eventually challenge the limits of breathable air, drinkable water, arable cropland and other limited resources. There's evidence that growth has been slowing in recent years and that world population could peak some time this century, then start to decline.

For this exercise, research world population growth issues online. *Be sure to investigate various viewpoints.* Get estimates for the current world population and its growth rate (the percentage by which it's likely to increase this year). Write a program that calculates world population growth each year for the next 75 years, *using the simplifying assumption that the current growth rate will stay constant*. Print the results in a table. The first column should display the year from year 1 to year 75. The second column should display the anticipated world population at the end of that year. The third column should display the numerical increase in the world population that would occur that year. Using your results, determine the year in which the population would be double what it is today, if this year's growth rate were to persist.

4.41 ***(Tax Plan Alternatives; The "FairTax")*** There are many proposals to make taxation fairer. Check out the FairTax initiative in the United States at

```
www.fairtax.org
```

Research how the proposed FairTax works. One suggestion is to eliminate income taxes and most other taxes in favor of a 23% consumption tax on all products and services that you buy. Some FairTax opponents question the 23% figure and say that because of the way the tax is calculated, it would be more accurate to say the rate is 30%—check this carefully. Write a program that prompts the user to enter expenses in various categories (e.g., housing, food, clothing, transportation, education, health care, vacations), then prints the estimated FairTax that person would pay.